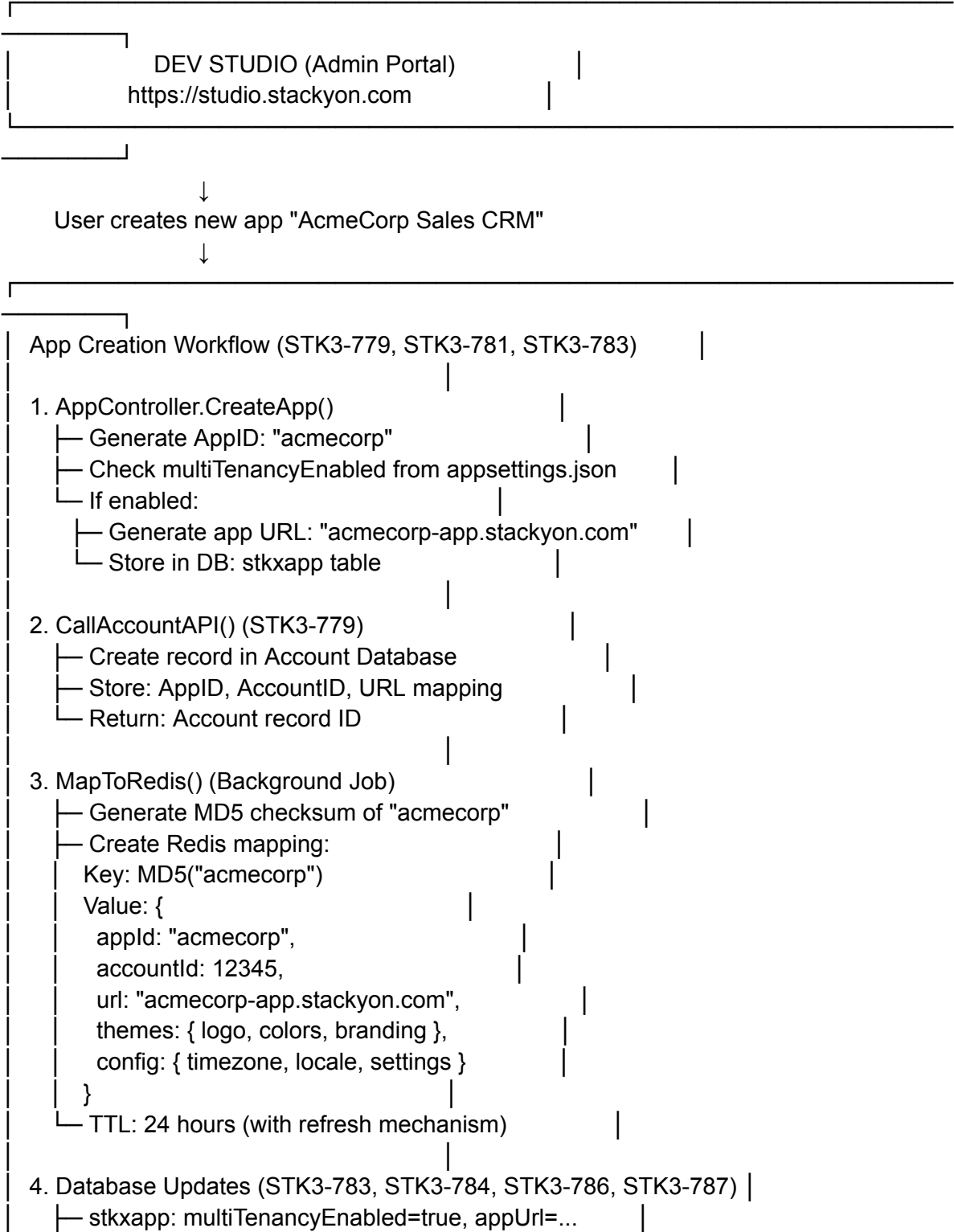


I'll provide you with a comprehensive implementation architecture showing how multitenancy flows from Dev Studio to the Internal Portal with AppID extraction.

🛠️ ****Complete Multitenancy Architecture: Dev Studio → Internal Portal****

****Phase 1: App Creation in Dev Studio****

...



```
stkxaccounts: url=...
stkxaccountthemes: appid, sourceType
stkxsourcetype: Add AccountThemes (Id=7)
```



```
Launch Button Displayed in UI
→ acmecorp-app.stackyon.com
```

...

Phase 2: User Access to Internal Portal

...

```
STEP 1: User Navigation
User clicks Launch or visits: https://acmecorp-app.stackyon.com
```



```
STEP 2: Frontend - AppID Extraction (STK3-807)
File: src/app/core/services/tenant.service.ts (NEW)

export class TenantService {
  private appId: string;
  private tenantConfig: ITenantConfig;

  extractAppIdFromUrl(): string {
    const hostname = window.location.hostname;
    // "acmecorp-app.stackyon.com"

    if (hostname.includes('-app.stackyon.com')) {
      const appId = hostname.split('-app.stackyon.com')[0];
      return appId; // "acmecorp"
    }
    return null; // Single-tenant mode
  }

  generateMD5Checksum(appId: string): string {
    return Md5.hashStr(appId).toString();
  }
}
```

↓

STEP 3: Account API Validation

POST /api/account/validate

Request:

```
{
  "checksum": "5f4dcc3b5aa765d61d8327deb882cf99",
  "appld": "acmecorp"
}
```

Response:

```
{
  "valid": true,
  "accountId": 12345,
  "appld": "acmecorp",
  "requiresAuth": true
}
```

↓

STEP 4: Redis Lookup (Backend)

Key: MD5("acmecorp")

Redis returns:

```
{
  "appld": "acmecorp",
  "accountId": 12345,
  "url": "acmecorp-app.stackyon.com",
  "themes": {
    "logo": "https://cdn.../acmecorp-logo.png",
    "primaryColor": "#007bff",
    "secondaryColor": "#6c757d",
    "favicon": "https://cdn.../favicon.ico",
    "loginBackground": "https://cdn.../bg.jpg"
  },
  "config": {
    "companyName": "Acme Corporation",
    "timezone": "America/New_York",
    "dateFormat": "MM/DD/YYYY",
    "language": "en-US",
    "termsUrl": "https://acme.com/terms"
  },
}
```

```
"features": {  
  "multiTenancyEnabled": true,  
  "ssoEnabled": false,  
  "customBranding": true  
}
```



STEP 5: Store Tenant Context in Frontend
File: src/app/core/services/tenant.service.ts

```
this.tenantConfig = response.data;  
sessionStorage.setItem('tenantConfig', JSON.stringify(...));  
this.tenantConfigSubject.next(this.tenantConfig);
```



STEP 6: User Authentication
Show Login Screen with Tenant Branding (STK3-791)

[Acme Corp Logo]

Welcome to Acme Corporation

Username:

Password:

[Login Button - Primary Color]

Background: Custom branded image



User enters credentials and clicks Login



STEP 7: Post-Authentication Validation (STK3-809)
File: src/app/core/security/auth.service.ts

After successful login:

1. Get userToken and decode user info
2. Extract user's associated applds from token
userInfo.apps = ["acmecorp", "techstart"]
3. POST /api/user/validate-tenant-access
Request:
{
 "userId": 789,
 "appld": "acmecorp",
 "sessionId": "xyz123"
}
4. Backend checks:
 - Is multiTenancyEnabled? (from config)
 - Does user belong to this appld?
 - Query: SELECT * FROM user_app_mapping
WHERE userId=789 AND appld='acmecorp'
5. Response:
{
 "accessGranted": true,
 "role": "admin",
 "permissions": ["read", "write", "manage"]
}
6. If accessGranted = false:
 - └ Show error: "You don't have access to this tenant"
 - └ Log security event
 - └ Redirect to error page or logout

↓
 Access Granted

STEP 8: Apply Tenant Branding & Load Application
Files: Multiple components (STK3-790, STK3-792, STK3-793)

1. App Component Initialization
 - Inject TenantService
 - Subscribe to tenantConfig\$
2. Dynamic Theme Application
document.documentElement.style.setProperty(
 '--primary-color',
 tenantConfig.themes.primaryColor
);

3. Update Favicon & Title

```
this.updateFavicon(tenantConfig.themes.favicon);  
document.title = tenantConfig.config.companyName;
```

4. Load User-Specific Data

- All API calls include: X-Tenant-ID: acmecorp
- Backend filters data by tenant context

5. Render Application with Tenant Context

[Acme Logo] Acme Sales CRM

Dashboard | Contacts | Reports

Welcome, John! (Acme Corporation)

[Tenant-specific content loads...]

Phase 3: HTTP Interceptor for Tenant Context

```
``typescript
```

```
// File: src/app/core/interceptors/tenant.interceptor.ts (NEW)
```

```
@Injectable()
```

```
export class TenantInterceptor implements HttpInterceptor {  
  constructor(private tenantService: TenantService) {}
```

```
  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
```

```
    const appld = this.tenantService.getAppld();
```

```
    const tenantConfig = this.tenantService.getTenantConfig();
```

```
    if (appld && tenantConfig?.features?.multiTenancyEnabled) {
```

```
      // Clone request and add tenant headers
```

```
      const clonedReq = req.clone({
```

```
        headers: req.headers
```

```
          .set('X-Tenant-ID', appld)
```

```
          .set('X-Tenant-Checksum', this.tenantService.getChecksum())
```

```
      });
```

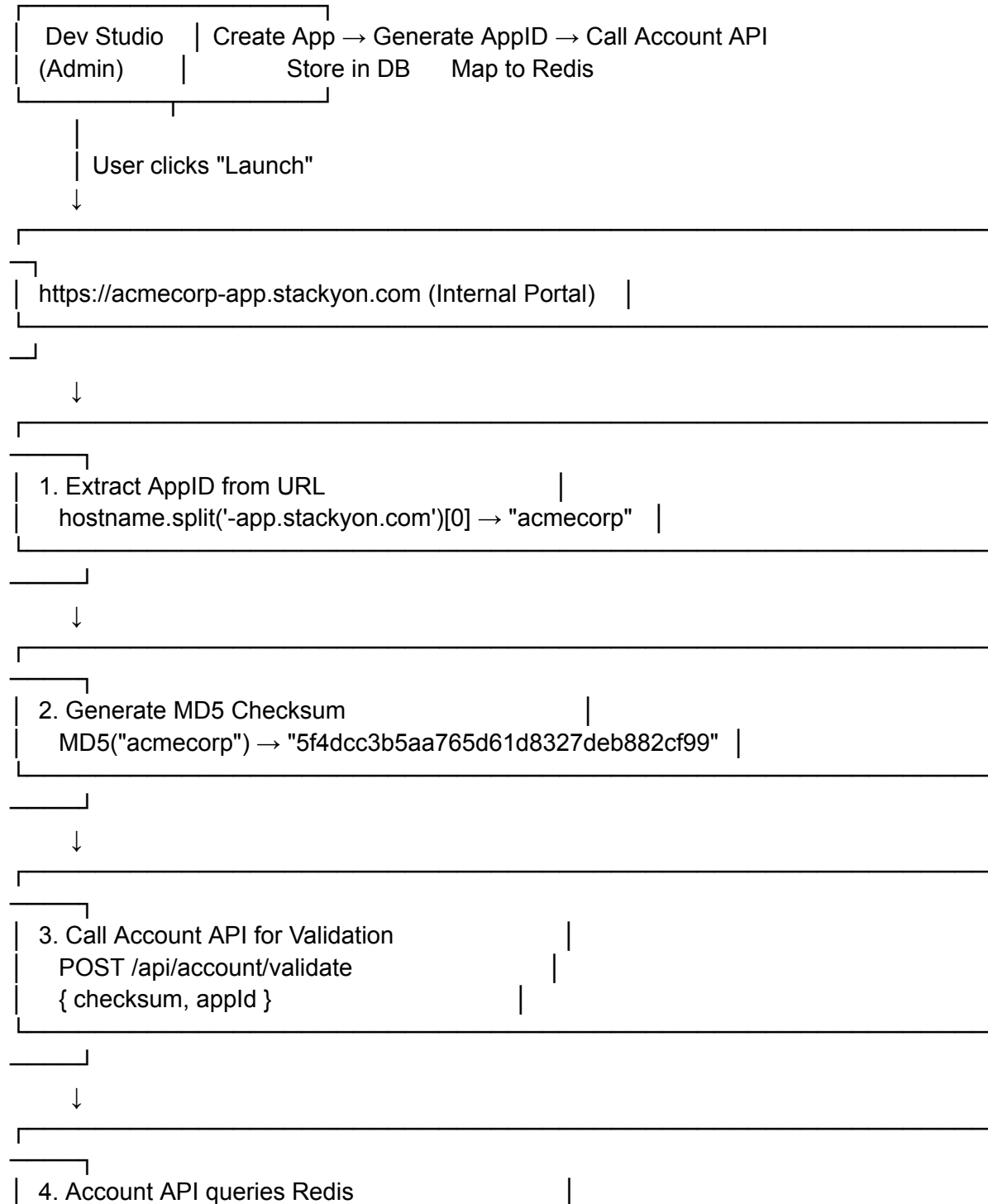
```
      return next.handle(clonedReq);
```

```
    }
```

```
    return next.handle(req);
  }
}
...
---
```

Complete Data Flow Diagram

...



Key: MD5 checksum
Returns: { appld, themes, config, features }



5. Frontend stores tenant context
SessionStorage + Service state



6. Show Branded Login Screen
Apply: logo, colors, background



User logs in



7. Post-Auth: Validate User-Tenant Association
POST /api/user/validate-tenant-access
Check: userId + appld mapping in DB



Valid?

Yes



No



Grant Access
Apply Branding
Load App

Deny Access
Show Error
Logout User



8. Application Running with Tenant Context

- All API calls include X-Tenant-ID header
- UI shows tenant branding
- Data filtered by tenant
- Config applied (timezone, locale, etc.)

Key Components & Files Structure

```
src/app/
├── core/
│   ├── services/
│   │   ├── tenant.service.ts (NEW - STK3-807)
│   │   │   ├── extractAppIdFromUrl()
│   │   │   ├── generateMD5Checksum()
│   │   │   ├── validateTenant()
│   │   │   ├── fetchTenantConfig()
│   │   │   ├── storeTenantContext()
│   │   │   └── getTenantConfig()
│   │   └── account-api.service.ts (NEW)
│   │       ├── validateChecksum()
│   │       └── fetchTenantData()
│   ├── interceptors/
│   │   └── tenant.interceptor.ts (NEW)
│   │       └── Add X-Tenant-ID header to all requests
│   ├── guards/
│   │   └── tenant-access.guard.ts (NEW - STK3-809)
│   │       └── canActivate() - validate user-tenant association
│   ├── models/
│   │   ├── tenant-config.interface.ts (NEW)
│   │   └── tenant-theme.interface.ts (NEW)
│   └── security/
│       ├── auth.service.ts (MODIFIED)
│       └── Add post-auth tenant validation
├── shared/
│   ├── components/
│   │   ├── branded-login/ (NEW - STK3-791)
│   │   ├── tenant-header/ (NEW)
│   │   └── theme-applicator/ (NEW - STK3-792)
└── app.component.ts (MODIFIED)
    └── Initialize tenant service on app load
```

...

Configuration Files

****Frontend: appsettings.json****

```
```json
{
 "multiTenancyEnabled": true,
 "accountApiUrl": "https://account-api.stackyon.com",
 "redisConfig": {
 "ttl": 86400
 },
 "tenantDefaults": {
 "theme": "default",
 "locale": "en-US"
 }
}
```
```

****Backend: `appsettings.json`****

```
```json
{
 "MultiTenancy": {
 "Enabled": true,
 "ValidateDomain": true,
 "RequireUserTenantMapping": true
 },
 "Redis": {
 "ConnectionString": "redis://...",
 "TenantCacheTTL": 86400
 },
 "AccountApi": {
 "BaseUrl": "https://account-api.stackyon.com",
 "ApiKey": "..."
 }
}
```
```

Database Schema Overview

```
```sql
-- STK3-783: stkxapp table
ALTER TABLE stkxapp ADD COLUMN multiTenancyEnabled BIT DEFAULT 0;
ALTER TABLE stkxapp ADD COLUMN appUrl NVARCHAR(200);
```

```

-- STK3-784: stkxaccounts table
ALTER TABLE stkxaccounts ADD COLUMN url NVARCHAR(200);

-- STK3-786: stkxaccountthemes table
ALTER TABLE stkxaccountthemes ADD COLUMN appid NVARCHAR(50);
ALTER TABLE stkxaccountthemes ADD COLUMN sourceType INT;

-- STK3-787: stkxsourcetype table
INSERT INTO stkxsourcetype (Id, Name) VALUES (7, 'AccountThemes');

-- User-Tenant Mapping (for STK3-809 validation)
CREATE TABLE user_app_mapping (
 Id INT PRIMARY KEY AUTO_INCREMENT,
 UserId INT NOT NULL,
 AppId NVARCHAR(50) NOT NULL,
 AccountId INT NOT NULL,
 Role NVARCHAR(50),
 CreatedDate DATETIME DEFAULT CURRENT_TIMESTAMP,
 UNIQUE KEY (UserId, AppId)
);

```

---

### ### \*\*Security Considerations\*\*

1. **\*\*MD5 Checksum Validation\*\***: Prevents URL tampering
2. **\*\*User-Tenant Association\*\***: Ensures users only access their assigned tenants
3. **\*\*Redis Cache\*\***: Fast lookup with automatic expiration
4. **\*\*HTTP Headers\*\***: All API requests include tenant context
5. **\*\*Route Guards\*\***: Protect routes from unauthorized tenant access
6. **\*\*Session Management\*\***: Tenant context stored in session, cleared on logout

---

### ### \*\*Error Handling Flow\*\*

...

Invalid AppID in URL

↓

Show error: "Invalid application URL"

↓

Redirect to default portal or error page

---

Valid AppID but not in Redis/DB

↓  
Show error: "Application not found"  
↓  
Log incident & notify admin

---

User not associated with tenant  
↓  
Show error: "Access denied to this application"  
↓  
Logout user & redirect to login  
...

---

This architecture provides a complete, secure, and scalable multitenancy implementation.  
Would you like me to start implementing any specific part using Speckit workflow?

### **Complete Multitenancy Architecture: Dev Studio → Internal Portal**

This comprehensive implementation architecture details how multitenancy flows from Dev Studio (the administrative application) to the Internal Portal (the user application), including the critical step of AppID extraction.-----Phase 1: App Creation in Dev Studio

The process begins in the **DEV STUDIO (Admin Portal)** at <https://studio.stackyon.com> when a user creates a new application, such as "AcmeCorp Sales CRM."

### **App Creation Workflow (STK3-779, STK3-781, STK3-783)**

1. **AppController.CreateApp()**:
  - Generates the AppID: "acmecorp".
  - Checks if `multiTenancyEnabled` is set in `appsettings.json`.
  - If enabled, it generates the app URL (`acmecorp-app.stackyon.com`) and stores it in the `stkxapp` table.
2. **CallAccountAPI() (STK3-779)**:
  - Creates a record in the Account Database.
  - Stores the AppID, AccountID, and URL mapping, returning the Account record ID.

3. **MapToRedis() (Background Job):**
  - Generates an MD5 checksum of the AppID ("acmecorp").
  - Creates a Redis mapping (Key: MD5 checksum) with a 24-hour TTL.
  - The value stores the full configuration: `appId`, `accountId`, `url`, `themes` (logo, colors, branding), and `config` (timezone, locale, settings).
4. **Database Updates (STK3-783, STK3-784, STK3-786, STK3-787):**
  - `stkxapp`: Sets `multiTenancyEnabled=true` and `appUrl`.
  - `stkxaccounts`: Updates the URL field.
  - `stkxaccountthemes`: Stores `appId` and `sourceType`.
  - `stkxsourcetype`: Adds `AccountThemes` (Id=7).

The process concludes with the Launch Button (pointing to `acmecorp-app.stackyon.com`) being displayed in the Dev Studio UI.-----Phase 2: User Access to Internal Portal

This phase details how a user accesses the newly created multitenant application and how the tenant context is established and secured.

1. **User Navigation (Step 1):** User visits the specific app URL, e.g., `https://acmecorp-app.stackyon.com`.
2. **Frontend AppID Extraction (Step 2 - STK3-807):**
  - The `TenantService` uses `extractAppIdFromUrl()` to parse the hostname (`acmecorp-app.stackyon.com`) and retrieve the AppID ("acmecorp").
  - An MD5 checksum is generated (`5f4dcc3b5aa765d61d8327deb882cf99`).
3. **Account API Validation (Step 3):**
  - A POST request to `/api/account/validate` sends the AppID and checksum to the backend.
  - The API confirms the validity of the tenant and that authentication is required.
4. **Redis Lookup (Step 4):** The backend uses the MD5 checksum as the key to query Redis, which returns the cached tenant configuration (`themes`, `config`, `features`).
5. **Store Tenant Context (Step 5):** The frontend `TenantService` stores the configuration locally in `sessionStorage` and updates its internal service state.
6. **User Authentication (Step 6 - STK3-791):** The Branded Login Screen is displayed, using the tenant's specific theme elements (logo, colors, custom background).
7. **Post-Authentication Validation (Step 7 - STK3-809):**
  - After successful login, the `auth.service.ts` extracts the user's associated AppIDs from the decoded token.
  - A POST request to `/api/user/validate-tenant-access` confirms the user is explicitly mapped to the requested `appId` in the `user_app_mapping` database table.
  - **Access Granted:** Continues to the application load.
  - **Access Denied:** Shows an error, logs a security event, and logs the user out.

## 8. Load Application (Step 8 - STK3-790, STK3-792, STK3-793):

- The App Component initializes the `TenantService`.
- Dynamic themes are applied (e.g., setting CSS `--primary-color`).
- Favicon and document title are updated.
- User-specific data loads, with all subsequent API calls automatically including the `X-Tenant-ID: acmecorp` header, ensuring backend data filtering by tenant context.

-----Phase 3: HTTP Interceptor for Tenant Context

The `TenantInterceptor` is crucial for ensuring that all API communication from the frontend includes the necessary multitenancy context headers, allowing the backend to serve tenant-specific data securely.

None

```
// File: src/app/core/interceptors/tenant.interceptor.ts
(NEW)

@Injectable()

export class TenantInterceptor implements HttpInterceptor {

 constructor(private tenantService: TenantService) {}

 intercept(req: HttpRequest<any>, next: HttpHandler):
 Observable<HttpEvent<any>> {

 const appId = this.tenantService.getAppId();

 const tenantConfig =
 this.tenantService.getTenantConfig();

 if (appId &&
 tenantConfig?.features?.multiTenancyEnabled) {

 // Clone request and add tenant headers
```

```

 const clonedReq = req.clone({
 headers: req.headers

 .set('X-Tenant-ID', appId)

 .set('X-Tenant-Checksum',
this.tenantService.getChecksum())

 });

 return next.handle(clonedReq);
 }

 return next.handle(req);
}
}

```

#### -----Key Components and Files Structure

The core implementation involves adding or modifying several files in the `src/app/core` directory:

- **src/app/core/services/**
  - `tenant.service.ts` (NEW - STK3-807): Core logic for AppID extraction, configuration handling, and checksum generation.
  - `account-api.service.ts` (NEW): Abstracts calls for tenant validation and fetching data.
- **src/app/core/interceptors/**
  - `tenant.interceptor.ts` (NEW): Automatically adds tenant headers (X-Tenant-ID, X-Tenant-Checksum) to outgoing HTTP requests.
- **src/app/core/guards/**
  - `tenant-access.guard.ts` (NEW - STK3-809): Implements a route guard to validate user-tenant association before allowing access to application routes.
- **src/app/core/security/**
  - `auth.service.ts` (MODIFIED): Modified to include post-authentication

tenant validation.

- **src/app/shared/components/**
  - **branded-login/** (NEW - STK3-791)
  - **theme-applicator/** (NEW - STK3-792)

#### -----Security Considerations

The architecture incorporates several layers of security:

- **MD5 Checksum Validation:** Prevents unauthorized URL tampering.
- **User-Tenant Association:** Ensures users are explicitly mapped to and can only access their assigned tenants.
- **HTTP Headers:** Every API request includes tenant context for secure, filtered data access.
- **Route Guards:** Protects front-end application routes against unauthorized tenant access.
- **Redis Cache:** Provides fast lookups and automatically expires tenant configuration data.
- **Session Management:** Tenant context is stored in the session and cleared upon user logout.

#### -----Error Handling Flow

Scenario	Error Message Displayed	Resolution
Invalid AppID in URL	"Invalid application URL"	Redirect to the default portal or error page.
Valid AppID but not in Redis/DB	"Application not found"	Log the incident and notify the admin.
User not associated with tenant	"Access denied to this application"	Logout the user and redirect to the login screen.